

Home Credit Default Risk Model Comparison

Model Development and Performance Review

Application Data Model Review | July 13, 2026

Executive Summary

This report reviews five classification methods for predicting repayment difficulty using the public Home Credit application data: Logistic Regression, Decision Tree, Random Forest, XGBoost, and LightGBM. The comparison uses the same data split and preprocessing framework for all models. The labeled sample contains 307,511 applications, with an observed event rate of 8.07 percent. Model performance is reviewed using AUC, AR, KS, PR-AUC, threshold metrics, risk-decile concentration, and probability calibration.

XGBoost produces the strongest overall test result, with an AUC of 0.7716 and KS of 0.4072. LightGBM is very close, with an AUC of 0.7702, while completing training much faster in the recorded run. Logistic Regression reaches an AUC of 0.7553 and remains a useful benchmark because its structure is easier to explain and maintain. The highest-risk XGBoost decile contains 35.13 percent of all observed bad cases, which confirms that the models provide useful risk ordering. The raw class-weighted boosting outputs should not be treated directly as default probabilities; Platt and isotonic calibration reduce the Brier score from about 0.185 to about 0.067.

For the current application-only dataset, LightGBM offers the best balance between performance and speed, while XGBoost remains the top-performing option by a small margin. Logistic Regression should be retained as the transparent benchmark. Before any production use, the model should be extended with customer history data, tested on later time periods, calibrated on a representative portfolio, and linked to approval, loss, and profitability analysis.

Contents

1	Project Scope and Data	3
1.1	Project objective	3
1.2	Target definition and sample profile	3
1.3	Data content and preparation	4
1.4	Data review points	4
1.5	Feature screening and scope	5
2	Model Overview	5
2.1	Logistic Regression	5
2.2	Decision Tree	6
2.3	Random Forest	7
2.4	XGBoost	7
2.5	LightGBM	8
3	Development Setup and Evaluation	8
3.1	Model settings and class imbalance	9
3.2	Ranking metrics	9
3.3	Threshold and portfolio metrics	10
3.4	Probability accuracy	10
4	Model Performance	10
4.1	Overall results	10
4.2	How to read close model results	11
4.3	Risk segmentation	12
4.4	Key drivers identified by the models	13
5	Probability calibration	14
6	Model Selection and Next Steps	15
6.1	Implementation considerations	15
6.2	Current limitations	16
6.3	Recommended next steps	16
7	Report Summary	17
A	Figure Sources and Reproducibility	17

1 Project Scope and Data

1.1 Project objective

The objective is to compare five commonly used classification methods under the same data and validation setup. The report focuses on four practical questions: how well each model ranks credit risk, how concentrated bad cases are in the highest-risk groups, whether the model output can be used as a probability, and whether the improvement from a more complex model is large enough to justify the additional work required to maintain it.

The current scope uses only `application_train.csv` from the Kaggle Home Credit Default Risk competition. Bureau records, previous applications, installment history, point-of-sale balances, and credit-card history are not included. The results therefore represent an application-data benchmark rather than a complete customer-level credit model.

1.2 Target definition and sample profile

The dependent variable is **TARGET**. A value of one denotes repayment difficulty under the competition definition, and zero denotes the absence of that outcome. The label is useful for comparative modeling, but it should not be treated as identical to an institution's regulatory default definition without a separate mapping exercise.

Table 1: Sample and feature counts

Item	Value
Applications	307,511
Non-events	282,686
Events	24,825
Observed event rate	8.07%
Original columns	122
Variables after cleaning and screening	108
Model inputs after one-hot encoding	232
Training / validation / test share	60% / 20% / 20%

The event rate is low enough that classification accuracy is not informative: a rule that assigns every case to the non-event class would already be correct for roughly 92 percent of observations. The report therefore emphasizes ranking and concentration measures, with threshold metrics used only after a cutoff has been selected on validation data.

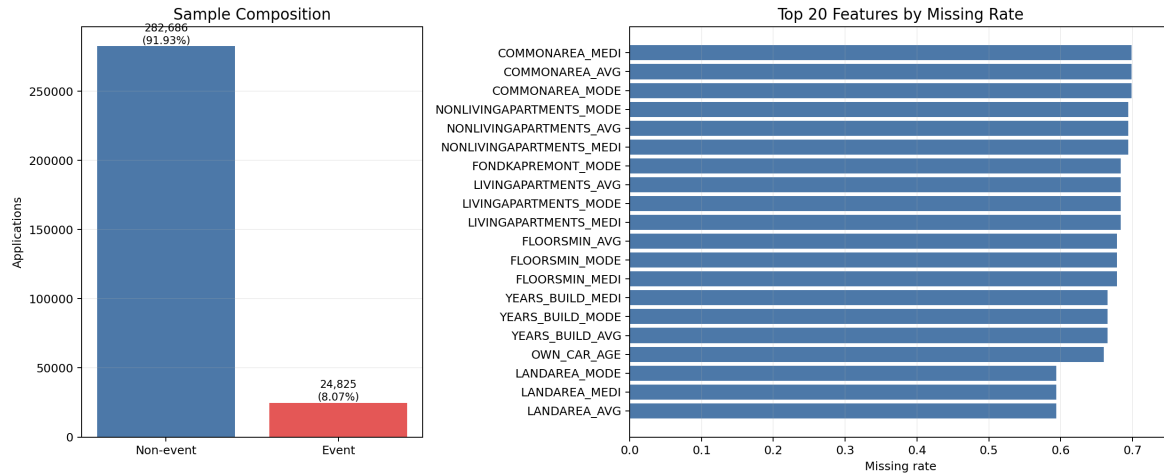


Figure 1: Sample composition and missingness in the application table. The class distribution motivates imbalance-aware evaluation, while missingness is concentrated in a subset of housing and external-information fields.

1.3 Data content and preparation

The application table combines contract terms, income and employment, family structure, housing and asset indicators, document flags, external credit scores, and recent bureau inquiry counts. The three anonymized `EXT_SOURCE` variables later emerge as the most stable predictors across models.

The main explicit anomaly is `DAYS_EMPLOYED=365243`, which appears 55,374 times and functions as a special code rather than a credible employment duration. It is replaced with a missing value before imputation. Numeric variables are imputed with training-set medians, categorical variables with training-set modes, and categorical levels are one-hot encoded. The Logistic Regression pipeline additionally standardizes numeric inputs. All preprocessing objects are fitted on training data only.

Seven ratio variables are constructed to express loan burden and household capacity: credit-to-income, annuity-to-income, credit-to-annuity, goods-price-to-credit, employment-to-age, income per household member, and payment rate. These transformations are simple, but they allow the model to distinguish, for example, a large loan requested by a high-income applicant from the same loan requested by a low-income applicant. `PAYMENT_RATE` and `CREDIT_ANNUIITY_RATIO` are reciprocal transformations and should not both be retained automatically in a final linear specification.

1.4 Data review points

The descriptive analysis focuses on patterns that directly support model development. External score distributions are compared by target outcome, repayment burden is reviewed by outcome, and event rates are summarized by age band. Income, credit amount, annuity, and goods price remain relevant for deeper segment review because their distributions are strongly right-skewed; medians, percentiles, or log-scale summaries are more informative than unadjusted means.

Categorical variables such as education, occupation, income type, family status, contract type, and organization type should be reviewed using both sample count and bad rate. A high bad rate in a very small category is not reliable on its own. Missing values should also be reviewed as possible signals rather than treated only as data quality issues, especially for external scores and housing variables.

The engineered ratio variables need basic range checks. Ratios can become unstable when the denominator is close to zero, and `PAYMENT_RATE` and `CREDIT_ANNUIITY_RATIO` contain essentially

the same information in reciprocal form. These points do not prevent the current model comparison, but they should be addressed before a final model is selected.

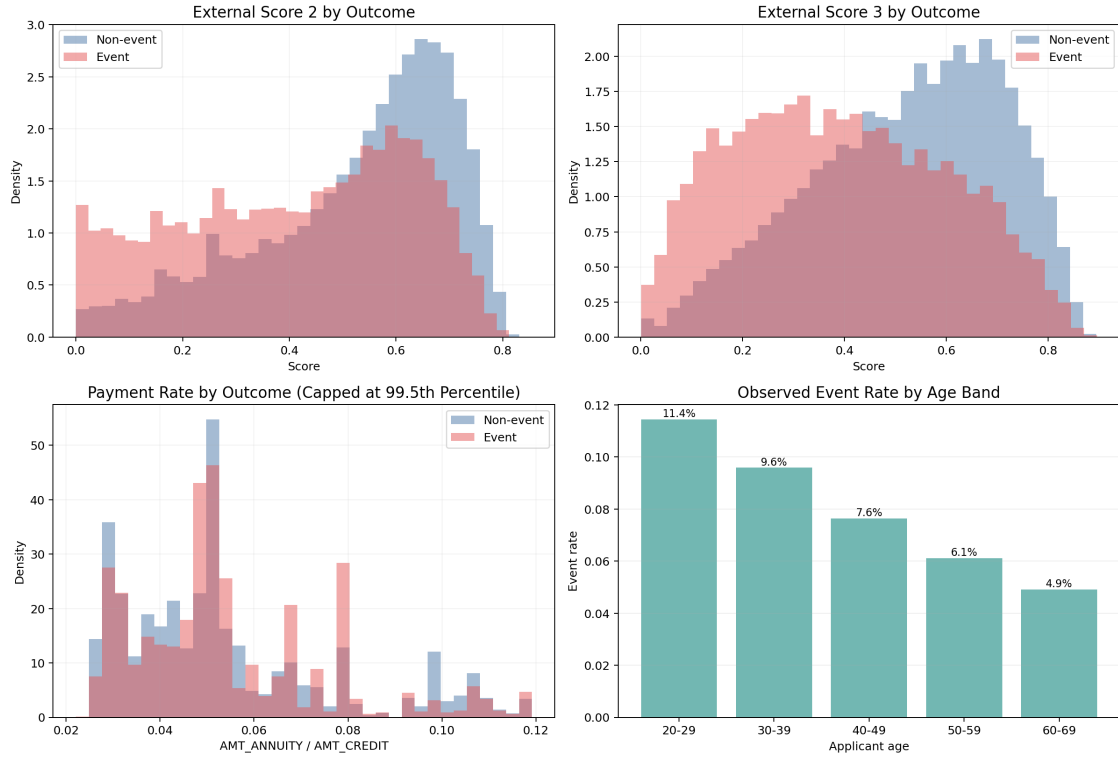


Figure 2: Selected variable distributions and observed event-rate patterns. External scores, repayment burden, and applicant age provide visible separation and motivate nonlinear model comparison.

1.5 Feature screening and scope

The notebook removes identifiers and the target from the predictor set, excludes unusable all-missing variables, and screens highly correlated numeric features using training data. The purpose is computational control rather than a claim that correlation screening identifies the economically correct specification. Correlation is a pairwise linear measure; it will not identify reciprocal variables, category combinations, or nonlinear redundancy. A final model would supplement this step with stability checks, missingness review, and a more explicit treatment of variables that may create policy or fairness concerns.

2 Model Overview

The five models use the same underlying application data but represent risk in different ways. The main differences are the type of relationship each model can learn, how overfitting is controlled, how much tuning is required, and how easily the result can be explained.

2.1 Logistic Regression

For applicant i , Logistic Regression models the log odds of the event as a linear function of the predictors:

$$\log\left(\frac{p_i}{1-p_i}\right) = \beta_0 + \sum_{j=1}^m \beta_j x_{ij}, \quad p_i = \frac{1}{1 + \exp[-(\beta_0 + x_i^\top \beta)]}. \quad (1)$$

The fitted model uses an L1 penalty,

$$\min_{\beta} \left\{ -\ell(\beta) + \lambda \sum_j |\beta_j| \right\}, \quad (2)$$

which can shrink weak coefficients to zero. Class weights are balanced so that events receive more influence than their raw frequency would imply.

The method is attractive in credit work because coefficients have a direction, score behavior is usually smooth, and implementation is straightforward. With standardized continuous inputs, coefficient magnitudes can also provide a rough within-model ranking of effects. These advantages are practical rather than cosmetic: validation, documentation, and adverse-action analysis are easier when the model is compact and directional.

The main limitation is its functional form. Unless transformations and interactions are added deliberately, the model assumes a linear relationship on the log-odds scale. Credit variables often have thresholds, flat regions, and interactions. Income may matter differently at different credit amounts, and the effect of employment duration may vary by age. L1 regularization does not solve this specification problem; it only controls the number and size of linear terms. Correlated one-hot variables can also make the selected coefficients unstable across samples even when aggregate performance is stable.

In this report Logistic Regression is the reference model. More complex methods are compared against it to determine whether the additional performance is large enough to justify the added tuning and implementation effort.

2.2 Decision Tree

A Decision Tree partitions the feature space recursively. At each node it selects a variable and threshold that reduce class impurity, here measured by the Gini criterion. For a node with event share q , Gini impurity is

$$G(q) = 1 - q^2 - (1 - q)^2 = 2q(1 - q). \quad (3)$$

The selected split maximizes the weighted reduction in impurity between the parent and its two children. The process continues until a stopping or pruning rule is reached, and the terminal-node event rate becomes the prediction for observations in that leaf.

The chief advantage is that nonlinearities and interactions arise naturally. A rule such as “external score below a threshold, high payment rate, and short employment history” does not have to be specified in advance. Upper-level splits are also easy to explain to nontechnical readers.

A single tree is, however, unstable. Small changes in the sample can change the selected split and alter a large part of the tree. Deep trees fit local noise, while heavily constrained trees can miss useful structure. Predicted probabilities are stepwise and may be based on relatively small terminal groups. The notebook limits depth and leaf size and applies cost-complexity pruning, but the resulting model is still mainly useful as an interpretable nonlinear benchmark rather than as the expected final model.

2.3 Random Forest

Random Forest averages many trees fitted to bootstrap samples. Each split considers only a random subset of predictors. For B trees, the probability estimate is

$$\hat{p}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x). \quad (4)$$

Bootstrap resampling and random feature selection reduce correlation among trees. Averaging then lowers variance relative to a single tree. Out-of-bag observations provide an internal diagnostic because each tree leaves out roughly one-third of the original cases.

Random Forest is usually robust to nonlinear relationships, interactions, monotonic transformations, and moderate noise. It requires less tuning than boosting and provides a useful challenger when the relationship between inputs and risk is not close to linear.

Its weaknesses are also visible in this experiment. The ensemble can fit the training data much more strongly than it performs out of sample, especially with many correlated predictors. Impurity-based feature importance is biased toward continuous variables and variables with many possible split points, and it has no sign. The model is harder to compress into a scorecard or a small set of rules. Random Forest probabilities may appear smoother than those of a single tree, but class weighting and leaf composition still affect calibration.

2.4 XGBoost

XGBoost builds trees sequentially. At iteration t , a new tree f_t is added to reduce the current loss:

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + \eta f_t(x_i), \quad (5)$$

where η is the learning rate. Using a second-order approximation to the loss, the method evaluates candidate splits from aggregated gradients and Hessians. The objective also penalizes tree complexity:

$$\mathcal{L}^{(t)} \approx \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \gamma K + \frac{1}{2} \lambda \sum_{k=1}^K w_k^2 + \alpha \sum_{k=1}^K |w_k|. \quad (6)$$

Row subsampling, column subsampling, depth constraints, minimum child weight, shrinkage, and early stopping provide additional regularization.

The method is well suited to tabular credit data because it can represent threshold effects and interactions without manual enumeration. It often performs well when predictors include mixtures of continuous variables, binary indicators, missing values, and engineered ratios. Gain-based importance and SHAP values can be used to examine the fitted model, although neither makes the model as simple as a linear scorecard.

The disadvantages are tuning cost, greater implementation complexity, and sensitivity to the validation design. A large search can overfit the validation set even when the final learner is regularized. Raw probabilities are also not guaranteed to be well calibrated, particularly when `scale_pos_weight` changes the effective class balance. In the executed notebook XGBoost gives the best rank ordering, but its training time is much higher than that of LightGBM.

2.5 LightGBM

LightGBM is also a gradient-boosted tree method. It bins continuous values into histograms and grows trees leaf-wise, choosing the leaf with the largest expected loss reduction. Histogram construction reduces memory and split-search cost. Leaf-wise growth can obtain a given training loss with fewer splits than level-wise growth, although it can also create deep local branches if leaf size and depth are not controlled.

The practical advantage is speed. On large, sparse, one-hot encoded tables, LightGBM can train much faster than conventional exact or level-wise boosting implementations. It retains the nonlinear and interaction capacity of boosted trees and supports regularization, column subsampling, row subsampling, and early stopping.

Its main risks are not fundamentally different from those of XGBoost. Leaf-wise growth can overfit small regions, split-count importance is not a measure of direction or economic contribution, and calibration needs to be assessed separately. In this notebook LightGBM has almost the same validation and test AUC as XGBoost. Because the performance gap is very small, the recorded speed difference becomes an important part of the model choice.

Table 2: Summary of model characteristics

Method	Main strength	Main limitation	Role in this comparison
Logistic Regression	Directional, compact, stable to implement	Linear log-odds form unless terms are engineered	Transparent benchmark
Decision Tree	Explicit nonlinear rules and interactions	High variance and coarse probabilities	Interpretable nonlinear reference
Random Forest	Variance reduction through averaging	Less transparent; can still overfit and importance is biased	Ensemble challenger
XGBoost	Strong control of complex tabular relationships	Tuning and computation; raw scores may be miscalibrated	Highest-performance candidate
LightGBM	Fast gradient boosting on large sparse data	Leaf-wise overfit risk; similar interpretation issues	Speed-performance candidate

How the Five Models Represent Risk

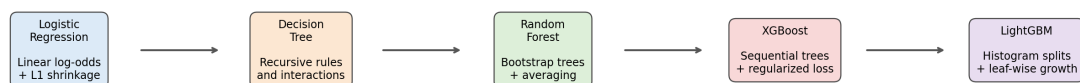


Figure 3: Conceptual comparison of the five model families and the fitted relationships they can represent from the same input data.

3 Development Setup and Evaluation

The sample is divided using stratified random splitting: 184,506 observations for training, 61,502 for validation, and 61,503 for testing. Each subset keeps an event rate close to 8.07 percent. Imputation, scaling, category encoding, and feature screening are fitted on the training sample. The validation

sample is used for parameter selection, early stopping, cutoff selection, and calibration choice. The test sample is reserved for the final comparison.

This design provides a consistent model comparison, but it is not an out-of-time validation. The results show performance on a held-out random sample, not performance on a later booking period.

3.1 Model settings and class imbalance

The five models are not forced to have the same structural complexity. Instead, each is given a parameterization intended to be plausible for the data and then evaluated under the same split. Logistic Regression uses an L1 penalty and balanced class weights. The tree is constrained by depth, minimum split size, minimum leaf size, and pruning. Random Forest uses 180 trees with depth and leaf constraints. XGBoost and LightGBM use small learning rates, regularization, subsampling, and early stopping.

Class weighting is used because an unweighted classifier can minimize loss largely by fitting the non-event majority. Weighting does not create new event observations; it changes their contribution to the objective. This generally helps the model pay attention to the minority class, but it also changes the raw probability scale. The calibration section therefore treats the weighted model output as a score until a probability mapping has been fitted.

Table 3: Parameterization used in the comparison

Model	Main settings
Logistic Regression	$C = 0.1$, L1 penalty, SAGA solver, balanced class weights, maximum 2,000 iterations.
Decision Tree	Maximum depth 8, minimum split 50, minimum leaf 500, 70 percent feature sampling, balanced weights, pruning alpha 10^{-5} .
Random Forest	180 trees, maximum depth 16, minimum split 150, minimum leaf 50, square-root feature sampling, balanced weights.
XGBoost	Learning rate 0.025, depth 4, minimum child weight 15, row subsample 0.75, L1 0.1, L2 10, gamma 0.1, early stopping.
LightGBM	Learning rate 0.04, 31 leaves, minimum child samples 80, column subsample 0.65, L1 0.5, L2 1, early stopping.

3.2 Ranking metrics

ROC-AUC measures the probability that an event observation receives a higher risk score than a non-event observation. It evaluates ordering over all possible thresholds, but it weights every part of the ROC curve equally even when only a narrow approval region is operationally relevant. Accuracy Ratio is a linear transformation,

$$AR = 2 AUC - 1, \quad (7)$$

so it does not add new information but expresses discrimination on a scale commonly used in credit risk work.

The KS statistic is the maximum difference between the cumulative event and non-event distributions:

$$KS = \max_c \{TPR(c) - FPR(c)\}. \quad (8)$$

KS identifies the point of greatest separation, not necessarily the economically preferred cutoff. PR-AUC focuses on precision and recall for the event class and is more sensitive than ROC-AUC to performance under low event prevalence.

3.3 Threshold and portfolio metrics

Precision is the proportion of flagged cases that are events, while recall is the proportion of all events captured. F1 is their harmonic mean. The notebook selects the maximum-F1 threshold on validation data and applies it unchanged to the test sample. This produces a consistent comparison, but F1 assigns no explicit cost to false approvals or false declines and therefore should not be interpreted as a lending objective.

Risk deciles provide a more portfolio-oriented view. Applications are sorted by predicted risk and divided into ten groups of similar size. Bad rate, cumulative event capture, and lift are then calculated by group. For the highest-risk decile,

$$Lift_{10} = \frac{\text{bad rate in the highest-risk 10 percent}}{\text{overall bad rate}}. \quad (9)$$

This shows how concentrated the adverse outcomes are in the population that a lender might review or restrict.

3.4 Probability accuracy

The Brier score is the mean squared difference between predicted probability and outcome,

$$Brier = \frac{1}{n} \sum_{i=1}^n (p_i - y_i)^2. \quad (10)$$

Log loss penalizes confident errors more heavily. Both measures depend on the probability scale rather than only the rank ordering. A model can therefore have high AUC and poor Brier score. This distinction is central in the present results because the weighted boosted models rank well before calibration but substantially overstate probability levels.

Leakage-Safe Development and Validation Flow



Figure 4: Leakage-controlled development and validation flow. Preprocessing and model fitting use the training sample; selection and calibration use validation data; the test sample is reserved for final measurement.

4 Model Performance

4.1 Overall results

Table 4: Final test-set results

Model	AUC	AR	KS	PR-AUC	Precision	Recall	F1	Brier	Log loss	Train sec.	Threshold
XGBoost	0.7716	0.5431	0.4072	0.2587	0.2584	0.4147	0.3184	0.1848	0.5478	1138.04	0.6659
LightGBM	0.7702	0.5404	0.4064	0.2555	0.2570	0.4127	0.3168	0.1849	0.5467	4.53	0.6672
Logistic Regression	0.7553	0.5107	0.3790	0.2314	0.2296	0.4272	0.2987	0.2023	0.5914	23.55	0.6477
Random Forest	0.7538	0.5075	0.3802	0.2290	0.2395	0.3960	0.2984	0.1736	0.5295	40.30	0.5686
Decision Tree	0.7231	0.4462	0.3247	0.2010	0.2037	0.4205	0.2745	0.2100	0.6058	2.33	0.6546

XGBoost ranks first on all four score-level discrimination measures. LightGBM follows closely. The AUC difference between them is 0.0014, which is too small to treat as a settled ordering without repeated resampling or paired confidence intervals. By contrast, the recorded training times differ by more than two orders of magnitude. Some of that gap may reflect implementation and parameter-search choices, but it is large enough to matter for iterative development.

Logistic Regression and Random Forest produce similar test AUCs. Their diagnostic patterns are different. Logistic Regression has training, validation, and test AUCs of 0.7476, 0.7479, and 0.7553. Random Forest reaches 0.8518 in training but only 0.7480 in validation and 0.7538 in testing. The forest therefore fits substantially more sample-specific structure without producing a corresponding out-of-sample gain. The single tree performs worst, which is consistent with the high variance and coarse partitioning expected from a constrained standalone tree.

The raw Random Forest probability measures are better than the raw boosting measures, even though its ranking is weaker. This is not contradictory. The positive-class weights used by XGBoost and LightGBM shift their score levels. Their uncalibrated outputs are useful as rankings but not as direct event probabilities.

Table 5: Training and validation AUC diagnostics

Model	Training AUC	Validation AUC	Test AUC	Train-validation gap
Logistic Regression	0.7476	0.7479	0.7553	-0.0003
Decision Tree	0.7372	0.7166	0.7231	0.0206
Random Forest	0.8518	0.7480	0.7538	0.1039
XGBoost	0.8137	0.7651	0.7716	0.0486
LightGBM	0.8189	0.7653	0.7702	0.0536

Logistic Regression shows very similar training, validation, and test AUC values. This does not confirm time stability, but it indicates limited overfitting within the current split. The Random Forest gap is the clearest warning sign. For the boosted models, a moderate training gap is expected because the ensembles are flexible; the more relevant observation is that validation and test AUCs remain close.

At the validation-selected maximum-F1 thresholds, the models reject roughly 13 to 17 percent of the test applications and capture about 40 to 43 percent of events. Logistic Regression obtains the highest recall, 42.72 percent, but flags a somewhat larger population. XGBoost captures 41.47 percent while producing higher precision and F1. These differences are conditional on the F1 rule. A lender targeting a fixed approval rate could obtain a different ordering at the operating point.

4.2 How to read close model results

The difference between XGBoost and LightGBM is very small: 0.0014 in AUC, 0.0008 in KS, and 0.12 percentage points in top-decile bad-case capture. These figures should be treated as a practical tie rather than as evidence that one algorithm is consistently superior. The current report compares the implemented versions of the models on one data split.

For a final selection, the close results should be checked using repeated splits or paired bootstrap samples on the same test observations. This would show whether the ranking is stable. The single Decision Tree also creates more tied scores because all cases in the same leaf receive the same prediction. That makes its curves and decile boundaries less smooth than those of the ensemble models.

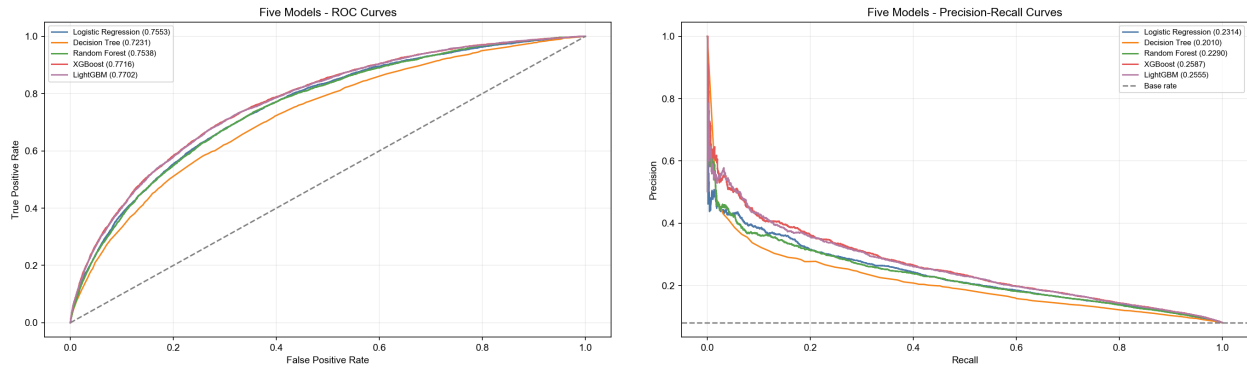


Figure 5: Test-sample discrimination for all five models. *Left:* ROC curves and AUC. *Right:* Precision-recall curves and PR-AUC; the boosted models are closely matched, while the single tree is weaker.

4.3 Risk segmentation

All five models produce increasing bad rates across the ten test-sample risk groups. The top-decile results are shown below.

Table 6: Highest-risk decile

Model	Share of all bad cases captured	Lift
XGBoost	35.13%	3.5122
LightGBM	35.01%	3.5001
Logistic Regression	32.73%	3.2725
Random Forest	32.15%	3.2141
Decision Tree	29.79%	2.9785

For XGBoost, the highest-risk 10 percent of applications contains slightly more than one-third of the observed events. With a portfolio event rate of 8.07 percent, a lift of 3.5122 corresponds to a top-decile event rate of roughly 28.3 percent. LightGBM is effectively the same on this measure. The difference between boosting and Logistic Regression is more visible in concentration than it is in the raw AUC numbers: the boosted models place about 2.3 to 2.4 additional percentage points of all events into the first 10 percent of cases.

This result is useful for risk segmentation, but it is not an approval policy by itself. A high-risk decile may be too large or too small for manual review, and a cutoff based on maximum F1 does not incorporate expected loss or income. The decile analysis only shows that the score contains useful ordering information.

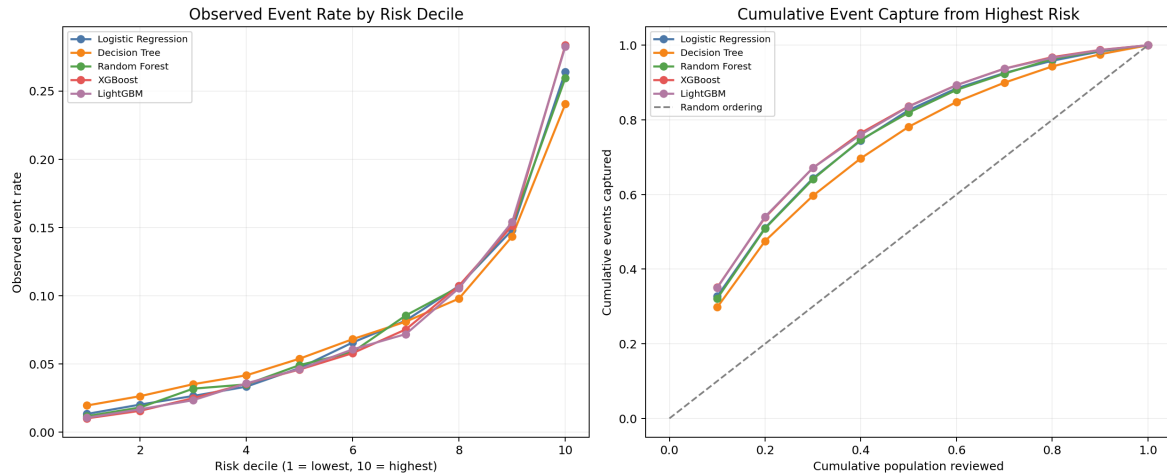


Figure 6: Test-sample risk segmentation. Observed event rates rise across score deciles, and cumulative capture shows the concentration achieved when applications are reviewed from highest to lowest predicted risk.

4.4 Key drivers identified by the models

The three external source variables rank highly across all model families. They have the largest absolute Logistic Regression coefficients and appear near the top of the tree-based importance rankings. The repeated appearance of these variables is more useful than comparing raw importance values, because each model calculates importance differently.

Repayment-burden variables also appear consistently. `PAYMENT_RATE`, `CREDIT_ANNUITY_RATIO`, and `GOODS_CREDIT_RATIO` are repeatedly selected by the boosted models and Random Forest. Age, employment duration, identification-document recency, and phone-change recency form a second group of frequently used variables. The pattern suggests that the model combines external credit information with loan structure and indicators of stability.

Interpretation remains limited in two respects. First, tree importance does not indicate whether a higher value raises or lowers risk. Second, a prominent variable can be a proxy for another variable or for a broader customer segment. A final analysis should use permutation importance and SHAP values on the same held-out sample, followed by dependence checks for the few variables that materially affect decisions.

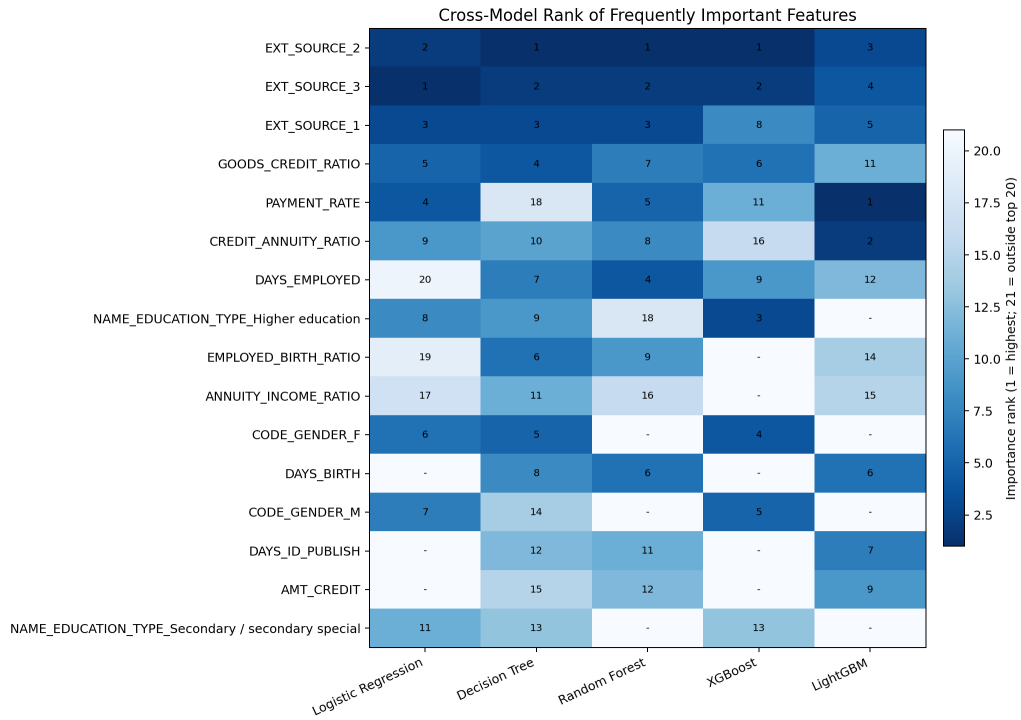


Figure 7: Cross-model rank heat map for frequently important features. Ranks are compared because raw coefficient and tree-importance magnitudes are not directly comparable across model families.

5 Probability calibration

Class weighting changes the effective balance used during training. As a result, the raw score level does not match the observed portfolio event rate. For both boosted models, the raw test Brier score is about 0.185. A constant forecast equal to the portfolio event rate would have a Brier score near 0.074, so the raw boosting probabilities are not acceptable as probabilities despite their good AUC.

The notebook compares Platt scaling and isotonic regression. Platt scaling estimates a smooth logistic mapping from model score to observed probability. Isotonic regression estimates a flexible monotonic mapping and can accommodate a non-sigmoid reliability curve, though it may produce flat sections and is more sensitive to calibration-sample size.

Table 7: Calibration results for the two leading models

Model	Mapping	Test AUC	Test KS	Test PR-AUC	Test Brier	Test log loss
LightGBM	Raw	0.7702	0.4064	0.2555	0.1849	0.5467
LightGBM	Platt	0.7702	0.4064	0.2555	0.0674	0.2433
LightGBM	Isotonic	0.7697	0.4063	0.2442	0.0673	0.2442
XGBoost	Raw	0.7716	0.4072	0.2587	0.1848	0.5478
XGBoost	Platt	0.7716	0.4072	0.2587	0.0672	0.2427
XGBoost	Isotonic	0.7713	0.4058	0.2489	0.0672	0.2433

Both mappings reduce probability error substantially while leaving ranking almost unchanged. Isotonic regression is selected in the notebook on validation Brier score, although the difference from Platt scaling is negligible. In a production use the simpler Platt mapping could be preferable if its stability is better across vintages. The practical conclusion is that ranking and probability estimation should be handled separately. A model can be selected for ranking performance and then calibrated on a representative sample.

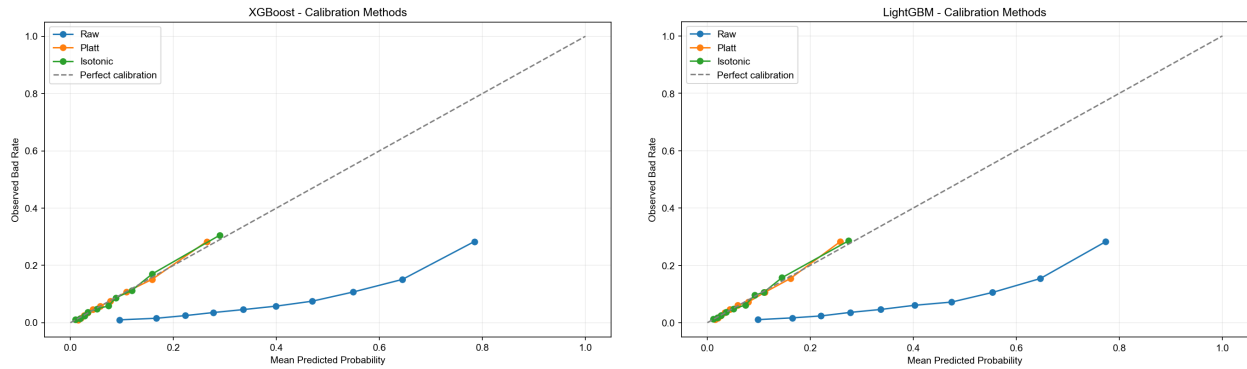


Figure 8: Test-sample reliability curves before and after probability calibration. *Left:* XGBoost raw, Platt-scaled, and isotonic probabilities. *Right:* LightGBM raw, Platt-scaled, and isotonic probabilities.

6 Model Selection and Next Steps

The comparison supports three practical conclusions. First, boosted trees improve risk ranking relative to the linear benchmark. Second, XGBoost and LightGBM perform almost identically on the test sample, so training speed and implementation preference should influence the choice. Third, Logistic Regression remains useful as a transparent benchmark because its performance is still reasonably close to the two leading models.

Table 8: Model choice by practical priority

Priority	Suggested model	Reason
Highest observed test discrimination	XGBoost	Best AUC, KS, PR-AUC, F1, and top-decile capture in this split.
Fast iteration with almost the same ranking	LightGBM	Test AUC is only 0.0014 lower, while recorded training time is far shorter.
Transparent benchmark and easier validation	Logistic Regression	Directional coefficients and very small train-validation gap.
Readable nonlinear rules	Decision Tree	Upper-level rules are easy to inspect, although standalone performance is weaker.
Independent nonlinear challenger	Random Forest	Useful as a different ensemble architecture, but the training gap requires attention.

A decision strategy would require more than a model ranking. Score cutoffs should be evaluated against approval rate, bad-case capture, expected loss, operating cost, and expected revenue. A simple expected-profit calculation could be written as

$$E[\Pi] = E[\text{interest and fee income}] - E[PD \times LGD \times EAD] - E[\text{funding and operating cost}]. \quad (11)$$

The maximum-F1 cutoff used in the notebook is useful for a standardized model comparison but is not a business optimum.

6.1 Implementation considerations

A production pipeline should preserve the training definitions exactly. Variable names, category levels, missing-value rules, ratio denominators, one-hot columns, model objects, and calibration mappings should be versioned together. Unknown categories at scoring time must have a documented treatment. The same applies to special values such as the employment-duration code. A model can fail operationally even when the fitted algorithm is sound if the production transforma-

tion differs from the development pipeline.

The score, calibrated probability, and credit policy should also remain separate objects. The score orders applicants. The calibration layer maps the score to an estimated event rate for a defined population and time period. The policy then combines that information with affordability, eligibility, fraud, documentation, and portfolio rules. Keeping these layers separate makes it possible to revise a cutoff or recalibrate a score without refitting the underlying rank-order model, provided its discrimination and stability remain acceptable.

For a boosted model, explanation should be generated from the deployed version rather than from a similar development fit. Global feature importance is useful for model review but is not enough for individual decisions. Local explanations should be checked for stability, and any reason-code process should avoid presenting a correlated proxy as a causal statement. Logistic Regression provides a useful comparison because its coefficient directions are explicit, but even there a one-hot coefficient is relative to its reference category and can change when correlated variables enter or leave the model.

Operational testing should cover more than average prediction latency. Batch scoring should be reconciled against notebook output on a fixed sample. Boundary cases, missing fields, unseen categories, extreme numeric values, and duplicated customer records should be included in unit tests. Score distributions should be compared before and after deployment, and a small set of cases should be manually traced through every transformation. These checks often identify scoring problems that are not visible in model-level metrics.

6.2 Current limitations

The analysis uses only the application table and therefore omits much of the customer's observed credit history. The split is random rather than out of time. The competition label may not match a bank's default policy. Parameter search uncertainty and sampling uncertainty are not reported. No fairness, prohibited-variable, or adverse-action review is included. Finally, there is no LGD, EAD, or revenue model, so the exercise cannot rank strategies by expected economic value.

6.3 Recommended next steps

The next extension should aggregate the relational tables by `SK_ID_CURR`: counts and status histories from bureau data, prior approval and rejection patterns, installment delinquency and payment ratios, credit-card utilization, and point-of-sale balance behavior. Aggregations should distinguish recency, frequency, severity, and trend rather than relying only on lifetime averages. For example, the number of recent bureau delinquencies, maximum installment delay, proportion of payments below the scheduled amount, recent rejection rate, and utilization trend may capture behavior that is absent from the application form.

The extended model should then be evaluated on later application vintages and by relevant segments. PSI alone is not enough. Monitoring should include feature missingness, score distribution, bad rate and sample count by score band, AUC/AR/KS by booking vintage, calibration drift, approval and decline rates, manual review volume, override rates and reasons, and realized expected loss. Segment reporting should be chosen before reviewing results and should reflect product, channel, geography, income type, and other operationally meaningful groups.

Model explanation should also be made more consistent. Permutation importance on the same test sample would allow a comparable global ranking across model classes. SHAP values could then be used for the selected boosted model, with dependence plots for the few variables that drive

material decisions. These tools do not remove governance concerns, but they are more informative than comparing incompatible built-in importance scales.

Recommended Extension and Validation Roadmap

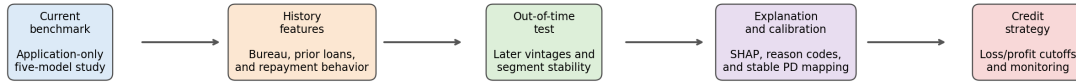


Figure 9: Recommended extension from the application-only benchmark to historical feature aggregation, out-of-time validation, consistent explanation, credit-strategy simulation, and ongoing monitoring.

7 Report Summary

The current benchmark shows that gradient-boosted trees provide the strongest risk ranking on the Home Credit application table. XGBoost records the highest test AUC, KS, PR-AUC, and top-decile capture, while LightGBM produces almost the same result with a much shorter recorded training time. Logistic Regression remains a strong reference model because it retains most of the available ranking power and is easier to explain and maintain.

From a portfolio perspective, the most useful result is the concentration of bad cases in the highest-risk groups. XGBoost and LightGBM place about 35 percent of observed bad cases in the highest-risk 10 percent of applications. This supports the use of the score for risk segmentation, review prioritization, and cutoff analysis. The cutoff itself should still be selected using approval rate, expected loss, operating capacity, and profitability rather than F1 alone.

The raw boosted-model outputs should be treated as risk scores until they are calibrated. The current calibration results are encouraging, but a final probability-of-default model would still require customer history data, out-of-time testing, segment-level stability checks, and a clear link between score bands and credit strategy.

A Figure Sources and Reproducibility

All figures in this report are produced by the project code. Sample composition and descriptive distributions use the public application table. Model-comparison, decile, and calibration figures use saved experiment outputs, with performance measured on the untouched test sample. Conceptual workflow figures document the implemented development logic and the recommended validation extension. The build step verifies that every required image is present before compiling the PDF.